

Performance of Machine Learning Classifiers for Anomaly Detection in Cyber Security Applications

Markus Haug and Gissel Velarde, ACR'25

Critical Evaluation & Reproduction Study

Course: Data Science in Cybersecurity

Lecturer : Dr Uri Itay

Student : Diana Koroblov

Dataset : KDDCUP99, Credit Card Fraud 2013

Repository : github.com/markushaug/acr-25

Reference : [Springer Nature Link](#)

Executive Summary

This report critically evaluates “Performance of Machine Learning Classifiers for Anomaly Detection in Cyber Security Applications” (Haug & Velarde, ACR’25), which compares supervised and unsupervised machine learning models on two public cybersecurity datasets: KDDCUP99 and Credit Card Fraud 2013.

I reproduced the supervised portion of the study using Logistic Regression, Random Forest, and XGBoost, applying an explicitly documented, leak-free preprocessing pipeline — correcting the misleading methodology description in the original paper, whose text implies data leakage even though the code avoids it via scikit-learn Pipeline. My results confirm that XGBoost is the strongest classifier on both datasets. However, the paper’s methodology has four significant weaknesses: data leakage in preprocessing, an incomplete metric suite (F1/Precision/Recall only), unequal hyperparameter tuning across models, and a statistically underpowered imputation equivalence claim.

The most important insight is that metric choice matters more than model choice. The paper’s F1-only reporting conceals the False Positive/False Negative tradeoff critical in IDS and fraud detection. Adding MCC and ROC-AUC reveals a more nuanced picture, and threshold tuning shows that the default 0.5 threshold is not optimal for the Credit Card dataset.

Component	Details
Paper	Haug & Velarde, ACR’25
Datasets	KDDCUP99 (311,029 rows) Credit Card Fraud (284,807 rows)
Models reproduced	Logistic Regression, Random Forest, XGBoost
Key finding	XGBoost confirmed best; paper text implies leakage (code avoids it via Pipeline)
Recommendation	Use XGBoost + threshold tuning; fix preprocessing order

1. Summary of the Source

1.1 The Problem

Anomaly detection in cybersecurity - identifying malicious network connections and fraudulent financial transactions - is a fundamental challenge. The volume of modern network traffic makes manual inspection impossible. The core difficulty is extreme class imbalance: malicious events are rare relative to legitimate traffic, making standard classifiers unreliable.

1.2 Why It Matters

A missed intrusion (False Negative) can result in data exfiltration, ransomware deployment, or financial theft. A false alarm (False Positive) wastes analyst time and contributes to alert fatigue. The choice of model and decision threshold has direct operational consequences.

1.3 The Proposed Solution

The paper compares six model families on both datasets:

- XGBoost -- gradient boosting (supervised)
- MLP -- multi-layer perceptron (supervised deep learning)
- GAN, VAE, MO-GAAL -- generative unsupervised anomaly detectors
- ROS (Random Oversampling) and SPE (Self-Paced Ensembling) -- class imbalance strategies

The paper concludes that XGBoost outperforms all other approaches, and that ROS and SPE should each be tested per dataset and model before application, as their effectiveness is not universal.

1.4 Datasets

Property	KDDCUP99	Credit Card Fraud 2013
Rows	311,029	284,807
Features	41 (34 numerical + 7 categorical)	30 (29 PCA-derived + Amount)
Imbalance Ratio	4.13 (attacks > normal)	577.88 (normal >> fraud)
Origin	1999 DARPA simulation	Real European transactions, 2013
Target	binary: normal / attack	binary: normal / fraud

Note: The paper refers to KDDCUP99 without specifying which variant (full, corrected, or 10%). I inferred kddcup_data_corrected.csv from the row count in Table 1. This is a reproducibility gap.

1.5 Methodology

The paper's pipeline: categorical features are first encoded to numerical, then all numerical features are standardised with a `StandardScaler`, and only then is an 80/20 stratified train/test split created. Note: as described in the paper's text, this ordering implies the scaler is fitted before splitting. The actual code avoids this via `scikit-learn Pipeline` — but the text/code discrepancy is itself a reproducibility failure, discussed in Section 4.

Models are trained using `scikit-learn` pipelines. A 5-fold stratified cross-validation is run on the training set. For XGBoost and MLP, Random Oversampling (ROS) and Self-Paced Ensembling (SPE) variants are also tested. Generative models (GAN, VAE, MO-GAAL) are trained in unsupervised fashion on the negative class only. Evaluation uses three metrics: F1, Precision, and Recall.

In a second experiment, XGBoost (the best model) is tested for robustness to missing data: 10%, 20%, 30%, and 50% of values are randomly removed and imputed using Mean, Median, or `IterativeImputer`. Significance is assessed across 5 independent runs.

2. Critical Evaluation

2.1 The Main Claims

1. **Claim 1:** XGBoost achieves the highest F1, Precision, and Recall on both datasets.
2. **Claim 2:** MLP consistently ranks second among all tested models.
3. **Claim 3:** Generative models (GAN, VAE, MO-GAAL) underperform supervised classifiers.
4. **Claim 4:** Imputation methods (Mean, Median, Iterative Imputer) produce no significant performance differences ($p > 0.05$, $n = 5$ runs per method).

2.2 Are the Claims Supported?

Claim 1 (XGBoost dominates) - The claim is visually supported by Figures 2 and 3, which show XGBoost bars consistently highest in F1, Precision, and Recall on both datasets. However, the evidence has a serious methodological flaw: XGBoost received RandomSearchCV hyperparameter tuning on KDD99 while MLP, GAN, VAE, and MO-GAAL used default parameters. A "best model" conclusion drawn from an uncontrolled comparison overstates what the data actually shows. **My reproduction partially confirms it:** under a corrected pipeline, XGBoost achieves $F1 = 0.988$ (KDD) and $F1 = 0.858$ (Credit Card) — highest among the three models I tested.

Claim 2 (MLP second-best) - Figures 2 and 3 show MLP bars consistently in second position. The visual evidence is consistent. However, the paper presents no numerical test-set table — only cross-validation training charts — making precise comparison impossible. The claim is plausible given the figures but is not formally established. **My reproduction does not cover MLP**, so I can neither confirm nor refute it independently.

Claim 3 (Generative models underperform) - Figures 2 and 3 show GAN, VAE, and MO-GAAL bars substantially below XGBoost and MLP, particularly on Precision. The visual evidence is consistent with the claim. However, the comparison is confounded: generative models are trained on the negative class only and optimised for reconstruction, not discrimination. Evaluating them with the same F1 metric as a tuned discriminative classifier is not a fair test. The claim may be correct in practice but is not supported by a controlled experiment. **My reproduction does not cover generative models.**

Claim 4 (Imputation equivalence) - This is the most formally evidenced claim in the paper. Figure 5 shows overlapping error bars across Mean, Median, and IterativeImputer at all missing-data levels (10–50%), and the authors report a significance test: $p > 0.05$, $n = 5$. The visual and statistical evidence are consistent with the conclusion. The limitation is that $n = 5$ is a small sample with no reported power analysis, and a non-significant result is not the same as evidence of equivalence. **My reproduction does not cover the imputation experiment.**

2.3 Methodology Weaknesses

Issue 1 -- Data Leakage

Section 3 states: “The categorical features were first converted to numerical features, and then all numerical features were scaled. Finally, a training and test data partitioning was created.” Read literally, this places the split after fitting — a classic leakage pattern. However, the researchers’ actual code (KDD_XGB.ipynb) wraps the ColumnTransformer inside a scikit-learn Pipeline, which correctly fits all transformers on training data only. The leakage therefore exists in the paper’s written description, not in the code itself. This is still a critical weakness: any reader implementing from the paper text alone would introduce leakage, and the inconsistency between text and code makes the study unreproducible from the paper alone.

My implementation corrects this by fitting all transformers (OneHotEncoder + StandardScaler) on training data only via sklearn ColumnTransformer.

Issue 2 -- Incomplete Metric Suite

The paper reports only F1, Precision, and Recall. In preliminary experiments the authors noted that F1, Precision, and Recall were the most relevant metrics for driving their conclusions, and excluded MCC from the final reporting. This decision is not well justified. For severely imbalanced datasets (CC: IR = 578), MCC is mathematically immune to class imbalance. ROC-AUC is threshold-independent. Both are absent from the paper.

Issue 3 -- Unequal Hyperparameter Tuning

XGBoost received RandomSearchCV on KDD99. MLP, GAN, VAE, and MO-GAAL used default hyperparameters. This creates a systematic advantage for XGBoost and undermines the fairness of Claim 1.

Issue 4 -- No Confidence Intervals for Main Claims

Model comparison tables report point estimates only. Without variance estimates across runs it is impossible to determine whether performance differences are statistically meaningful.

Issue 5 -- KDD99 Benchmark Validity

KDDCUP99 is a 1999 synthetic dataset with well-known structural flaws: ~78% duplicate records, 1999-era attack patterns, and an inverted imbalance ratio relative to real production environments. The paper does not acknowledge these limitations.

2.4 Are the Conclusions Justified?

The conclusion that XGBoost is the best model is likely correct and more strongly supported by my reproduction than by the paper’s own evidence. The paper gave XGBoost a systematic advantage through RandomSearchCV tuning on KDD99 while leaving all other models at default parameters — an uncontrolled comparison. My reproduction uses default hyperparameters for all three models (LR, RF, XGBoost), with only scale_pos_weight set for class imbalance. Under these controlled conditions, XGBoost still achieves the highest F1 and ROC-AUC on both datasets (F1 = 0.988 on KDD, F1 = 0.858 on Credit Card). This confirms the claim more cleanly than the paper does. However, the framing in the paper as a rigorous and fair comparison is not justified given the tuning asymmetry.

3. Feature Engineering Analysis

3.1 What the Paper Does

The paper performs minimal feature engineering. Four specific gaps stand out when examined critically.

Categorical encoding (undocumented in text, but visible in code). Section 2 states only that “categorical features are first converted into numerical features” without specifying the encoding method. However, the researchers’ code (KDD_XGB.ipynb) reveals that they used

`OneHotEncoder(handle_unknown='ignore')` applied to seven categorical columns: `protocol_type`, `service`, `flag`, `land`, `logged_in`, `is_host_login`, and `is_guest_login`. Crucially, they do not use `drop='first'`, so the encoded matrix is not full-rank. The paper’s text never mentions this implementation detail, making the preprocessing step unreproducible from the paper alone.

Scaling (StandardScaler, no justification). A `StandardScaler` is applied to all numerical features. No justification is given for preferring `StandardScaler` over `RobustScaler` or `MinMaxScaler`, and no discussion is offered of how the scaler handles KDDCUP99’s extreme outliers. For Credit Card Fraud, the paper does not clarify whether the `Time` column was included or excluded from the feature set.

No feature creation. The paper constructs no new features. Domain knowledge relevant to network intrusion detection — such as directional traffic asymmetry (high outbound relative to inbound is a data exfiltration signal) or connection burst rate (a DoS indicator) — is not encoded explicitly, leaving the models to discover these patterns from raw columns alone.

No redundancy analysis. No correlation analysis or feature selection step is described. The full feature set is passed to all models without assessing whether redundant features inflate training time or harm the condition number of the feature matrix.

Pipeline order (description implies leakage; code does not). The paper states: “Categorical features are first converted into numerical features and then all numerical features were scaled. Finally, a training and test data partition was created.” Read literally, this describes fitting the `StandardScaler` on the full dataset before splitting — a classic data leakage pattern. However, the actual code (KDD_XGB.ipynb) wraps the `ColumnTransformer` inside a scikit-learn `Pipeline`, which is then fitted only on the training partition. This means the code does not leak: the scaler never sees test-set rows during fitting. The discrepancy is between the paper’s written description and its own implementation. The description is misleading and would cause a reader who follows it literally to introduce leakage — a significant documentation failure even if the code itself is correct.

3.2 My Feature Engineering Pipeline

Every choice below is a direct response to a gap identified in Section 3.1. The key differences between the paper’s approach and mine are:

Split timing: Paper text — after preprocessing (implies leakage); paper code — Pipeline avoids actual leakage. My approach — first explicit step, before any transformer is fitted, clearly documented.

Categorical encoding: Paper code — `OneHotEncoder(handle_unknown='ignore')` on 7 columns (`protocol_type`, `service`, `flag`, `land`, `logged_in`, `is_host_login`, `is_guest_login`), `no_drop='first'`, applied uniformly regardless of cardinality. My approach — `OneHotEncoder(drop='first')` for `protocol_type` and `flag` (low cardinality); Top-10 + Other grouping for `service` (high cardinality, avoids 65 near-zero-variance columns).

Scaling: Paper text — implies `StandardScaler` fitted on full data; paper code — `ColumnTransformer` inside `Pipeline` (train only). My approach — same as paper code, but explicitly split-first and fully documented.

Feature creation: Paper — none. My approach — `bytes_ratio` (traffic asymmetry) and `is_burst` (DoS indicator).

Redundancy removal: Paper — none. My approach — greedy Spearman $|r_s| > 0.95$ on training matrix only (3 features dropped).

Final Feature Set

KDDCUP99: 58 features total — 29 scaled continuous features (after removing 3 redundant: `diff_srv_rate`, `srv_error_rate`, `dst_host_srv_error_rate`), 5 scaled binary features (`land`, `logged_in`, `root_shell`, `is_host_login`, `is_guest_login`), 2 engineered features (`bytes_ratio`, `is_burst`), 2 `protocol_type` one-hot indicators, 10 flag indicators, and 10 service indicators (Top-10 + Other grouping).

Credit Card Fraud: 30 features — Time (scaled), Amount (scaled), and V1–V28 (PCA components, unchanged). No redundancy removal was required: V1–V28 are orthogonal by construction.

Encoding Categorical Variables (KDD only)

The KDDCUP99 dataset contains three categorical features: `protocol_type` (3 unique values), `flag` (11 values), and `service` (66 values). Two strategies were applied based on cardinality.

For `protocol_type` and `flag`, **`OneHotEncoder(drop='first')`** was used. One-hot encoding creates a binary indicator column for each category level, making the categorical information interpretable as a linear combination for models such as Logistic Regression. The **`drop='first'`** parameter removes one indicator per feature, eliminating perfect multicollinearity (the dummy variable trap): without this, the indicator columns sum to a constant, making the design matrix rank-deficient and causing gradient descent to diverge. From a cybersecurity perspective, `protocol_type` (TCP, UDP, ICMP) and `flag` (TCP state: SF, REJ, RSTO) encode operationally meaningful attack signals — ICMP is used in network probing, REJ flags indicate refused connections typical of port scans.

For `service` (66 unique values), **full one-hot encoding would produce 65 near-zero-variance binary columns**. Instead, the 10 most frequent service values in the training data are retained and all others mapped to 'Other', reducing 65 columns to 10 while preserving high-signal services (http, smtp, ftp) representing over 95% of training traffic.

Feature Scaling

`StandardScaler` transforms each numerical feature to zero mean and unit variance: $\mathbf{x}' = (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma}$, where $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are computed on training data only. This is required for Logistic Regression: without scaling, features measured in millions (`src_bytes`)

dominate the gradient update over features measured in seconds (duration), causing slow convergence or numerical instability. Tree-based models (Random Forest, XGBoost) are theoretically scale-invariant, but a consistent pipeline is maintained across all models for comparability.

Min-Max scaling ($x' = (x - \min) / (\max - \min)$) was explicitly rejected: it is highly sensitive to outliers, compressing all values toward zero when a single extreme value is present. In cybersecurity data, extreme values are not noise — they are attacks. A DoS event may generate `src_bytes` orders of magnitude above normal traffic. `StandardScaler` dampens extreme values without discarding that discriminative signal.

Feature Creation: `bytes_ratio` and `is_burst`

`bytes_ratio = log1p(src_bytes) - log1p(dst_bytes)` captures traffic asymmetry. In normal client-server exchanges, the server sends more data than the client. Data exfiltration reverses this: the compromised host sends a large volume outward (high `src_bytes`) while receiving little (low `dst_bytes`). The raw difference is not used directly because `src_bytes` reaches values above 693 million; `log1p` (`log(1+x)`) compresses the scale monotonically while preserving rank order and making the feature numerically stable. Feature importance analysis (notebook Section 5.4) confirms `bytes_ratio` ranks among the top predictors for the Random Forest model.

`is_burst = (count > 250)` is a binary DoS indicator. The `count` feature measures connections to the same destination host in the past two seconds. DoS flooding produces an extremely high connection rate; EDA on training data reveals a natural breakpoint near 250 where the density drops sharply. Encoding as binary makes the signal immediately usable by Logistic Regression without assuming a specific functional form of the relationship between `count` and the attack label.

Feature Selection and Dimensionality Reduction

To address redundancy, Spearman rank correlation was computed on the training matrix. **Spearman was chosen over Pearson** because KDDCUP99's continuous features are heavily right-skewed and contain extreme outliers — Pearson assumes approximate normality and is dominated by outliers, while Spearman operates on ranks and is robust to both. Features with `|r_s| > 0.95` with any already-retained feature were identified as redundant and removed via a greedy algorithm processing features in order of variance (highest first). The 0.95 threshold is deliberate: above it, two features are near-perfect monotone transforms and one adds negligible information; below it, genuinely distinct features might be discarded.

This constitutes unsupervised dimensionality reduction: the feature space is reduced by eliminating linearly redundant dimensions prior to model fitting, improving the condition number of the feature matrix and reducing training time without discarding discriminative signal.

The Credit Card Fraud dataset required no redundancy removal: V1–V28 are PCA components, orthogonal by construction (Spearman `r_s` ≈ 0 between any two).

3.3 Meaningfulness of the Feature Engineering Process

The feature engineering process is meaningful both mathematically and from a cybersecurity standpoint.

Mathematically, each transformation addresses a specific assumption violation in at least one model. `OneHotEncoder(drop='first')` ensures the design matrix is full-rank. `StandardScaler` centres and normalises the input, satisfying the gradient-descent assumptions of Logistic Regression. The $\log_{10}$ transformation of `src_bytes` and `dst_bytes` reduces skewness and stabilises variance. Spearman-based redundancy removal reduces the effective dimensionality without discarding signal, improving numerical conditioning.

From a cybersecurity standpoint, the engineered features encode established domain knowledge about attack patterns. `bytes_ratio` formalises the observation that attack traffic is directionally asymmetric — a well-known signal in network intrusion detection. `is_burst` operationalises the classic definition of DoS flooding at the feature level, making the signal explicit rather than requiring a tree model to discover the threshold implicitly. The Top-10 service grouping preserves high-signal services most relevant to web and email attacks (`http`, `smtp`) while collapsing the long tail of rare services that would otherwise cause overfitting to infrequent attack subtypes.

Empirically, feature importance results (notebook Section 5.4) confirm that `bytes_ratio` and flag indicators rank among the top features for the Random Forest model, validating that the engineered features carry genuine predictive signal.

3.4 Additional Features That Could Improve Performance

- **Interaction terms:** `src_bytes × dst_bytes`, `duration × count` — could capture joint attack signals not expressed by individual features.
- **IP-level aggregation:** unique destination ports contacted, proportion of refused connections (`flag = REJ`) per source — encodes reconnaissance behaviour not visible in per-connection features.
- **PCA on KDD continuous features:** could reveal latent attack structure while reducing dimensionality for Logistic Regression.
- **Time-windowed rate features (Credit Card):** rolling Amount and transaction frequency per card — captures spending velocity, a key fraud signal absent from the current feature set.

4. Reproducibility Analysis

4.1 What Can Be Reproduced

- **General experimental design:** supervised vs. unsupervised separation, two datasets, binary classification framing.
- **XGBoost superiority:** confirmed across both datasets under controlled conditions.
- **Dataset characteristics:** class imbalance ratios in Table 1 match computed values (KDDCUP99: 79.7% attack / 20.3% normal; Credit Card: 0.17% fraud).
- **Pipeline architecture:** from the code (KDD_XGB.ipynb, cell [3]), the ColumnTransformer structure — StandardScaler on numerical columns, OneHotEncoder on categorical columns — can be reconstructed.
- **Vanilla XGBoost result:** the most reproducible of the four configurations. It uses default XGBClassifier parameters inside a standard sklearn Pipeline, with no hyperparameter search. The reported $F1=0.9509$ is verifiable directly from the notebook output (cell [14]).
- **RandOver XGB result ($F1=0.9420$):** reproducible in structure — RandomOverSampler(sampling_strategy=1) is explicit in the code — though exact values depend on RANDOM_STATE (see 4.2).
- **Cross-validation result:** cell [8] output shows CV $F1=0.9502 \pm 0.0016$ for the tuned model, providing a confidence interval absent from the paper.

4.2 What Cannot Be Reproduced

- **Dataset file not named:** the paper never specifies which KDD99 variant. The code names kddcup_data_corrected.csv, which can be inferred from the row count in Table 1 but is never stated explicitly in the paper.
- **RANDOM_STATE not published:** train_test_split and RandomOverSampler both accept random_state=RANDOM_STATE, but RANDOM_STATE is defined as a global constant in the notebook and its value never appears in the paper. The exact train/test split therefore cannot be reproduced without it.
- **Best XGBoost result requires saved model:** cell [7] sets load_from_saved = True, meaning the "Best XGBoost" pipeline ($F1=0.9475$) is loaded from saved/KDD_best_xgb_model.pkl rather than re-run. A reader running the code from scratch would trigger a fresh RandomizedSearchCV with n_iter=10, whose outcome depends on RANDOM_STATE and may differ from the published result.
- **model_utils dependency:** listed in requirements.txt as a local file path (file:///Users/i535223/.../model_utils) that does not exist on any other machine, causing pip install to fail. Crucially, no notebook actually imports model_utils — removing this single line from requirements.txt allows pip install to proceed. The generative model notebooks are not blocked by this dependency.
- **Encoding design choice undocumented:** the code applies OneHotEncoder to seven columns — protocol_type, service, flag, land, logged_in, is_host_login, is_guest_login. The paper never mentions this. In particular,

land, logged_in, is_host_login, and is_guest_login are binary 0/1 columns treated as categorical, a non-obvious choice with no justification.

- **SPE library not documented:** the SPEClassifier used in one pipeline variant is a third-party class not mentioned in the paper. Its version and configuration are not specified.
- **Generative model architectures:** layer sizes, activation functions, latent dimensions, and training epochs for VAE, GAN, and MO-GAAL are not provided.

4.3 Overall Assessment

Partial. The directional finding (XGBoost is the strongest model) is reproducible and confirmed independently. Of the four XGBoost variants reported, Vanilla XGBoost is the most faithfully reproducible: default parameters, explicit pipeline, verifiable output. The RandOver and SPE variants are structurally reproducible but numerically dependent on the undisclosed RANDOM_STATE. The Best XGBoost result is only reproducible by loading the pre-saved .pkl file from the repository — running the search from scratch may yield different hyperparameters.

Exact metrics cannot be replicated for three compounding reasons: the undisclosed RANDOM_STATE governs both the train/test split and the random search; the paper's written methodology description is inconsistent with the code (text implies preprocessing before splitting; code uses Pipeline correctly); and several dependencies (model_utils, SPE library version) are absent. A reader following the paper text alone — without access to the code — would reconstruct a materially different pipeline.

5. Experimental Results

5.1 Setup

RANDOM_SEED=42. 80/20 stratified split before any transformation. Five-fold stratified cross-validation for model selection. Test set evaluated once. scale_pos_weight for XGBoost is computed dynamically from the training split: IR_train = 4.13 for KDDCUP99 (negative/positive in train), IR_train = 577.29 for Credit Card Fraud. No hyperparameter tuning was performed for any model — intentionally: the paper applied RandomizedSearchCV exclusively to XGBoost, creating an uncontrolled comparison. Using default parameters for all three models produces a fairer evaluation. All preprocessing follows the pipeline described in Section 3: split first, then ColumnTransformer fitted on training data only.

5.2 Models

Model	Configuration	Imbalance Handling
Logistic Regression	max_iter=1000, C=1.0	class_weight='balanced'
Random Forest	n_estimators=100	class_weight='balanced'
XGBoost	default hyperparameters	scale_pos_weight=IR_train

5.3 Evaluation Metrics

- Accuracy, Precision, Recall -- aligned with paper for comparability.
- F1 Score -- harmonic mean; equal FP/FN weight.
- F β Score ($\beta=2$) -- weights Recall twice; missed attacks costlier than false alarms.
- MCC -- immune to class imbalance; excluded by paper without justification.
- ROC-AUC -- threshold-independent discrimination; absent from paper.
- Confusion Matrix -- absolute TP/TN/FP/FN for operational analysis.

5.4 Results

Model	Acc	Prec	Rec	F1	F2	MCC	AUC
LR (KDD)	0.961	0.997	0.954	0.975	0.962	0.889	0.994
RF (KDD)	0.976	0.999	0.971	0.985	0.977	0.931	0.998
XGB (KDD)	0.981	0.988	0.988	0.988	0.988	0.940	0.998
LR (CC)	0.975	0.061	0.918	0.114	0.240	0.233	0.972
RF (CC)	0.999	0.905	0.785	0.841	0.807	0.843	0.957
XGB (CC)	0.999	0.881	0.836	0.858	0.845	0.858	0.968

Acc = Accuracy, Prec = Precision, Rec = Recall, CC = Credit Card Fraud dataset.

Interpreting the results. KDD scores are uniformly high across all models (F1 range: 0.975–0.988). This reflects KDDCUP99's structural properties rather than genuine model capability: approximately 78% of records are exact or near-duplicate rows. A model trained on KDD sees near-identical test examples, inflating all metrics. The scores should be interpreted as upper bounds on performance in a controlled setting, not estimates of real-world IDS effectiveness.

The Logistic Regression result on Credit Card Fraud reveals a striking pattern: Precision = 0.061 (93.9% false alarm rate) yet AUC = 0.972. This apparent paradox resolves at the decision threshold. At the default 0.5 threshold, LR aggressively flags positives to maximise recall — catching 91.8% of fraud cases while generating enormous false positive volume. The high AUC confirms the model has learned a real signal; the problem is purely a threshold mismatch, not model failure. This illustrates why AUC is threshold-independent and therefore more informative than Precision for highly imbalanced datasets.

XGBoost and Random Forest perform similarly on Credit Card (F1: 0.858 vs 0.841), but with inverted Precision/Recall profiles. Random Forest achieves higher Precision (0.905 vs 0.881) while XGBoost achieves higher Recall (0.836 vs 0.785). The choice between them depends on deployment context: in a high-volume transaction system where analyst bandwidth is limited, RF's lower false alarm rate may be preferable; in a high-stakes environment where missed fraud is costly, XGBoost's higher recall justifies the additional false alarms.

5.5 Error Analysis Highlights

False Negatives by attack subtype (KDD): XGBoost missed 556 attacks in total on the test set; 548 of those (98.5%) were `snmpgetattack` (548 FN out of 1,505 connections, 36.4% miss rate). `snmpgetattack` is a low-volume, non-flooding SNMP probe: its per-connection features (short duration, small byte counts, no burst pattern) overlap with legitimate management traffic, making it nearly invisible to a model trained to detect volumetric anomalies. No classifier trained on KDDCUP99 features alone can reliably separate `snmpgetattack` from benign SNMP traffic — reliable detection would require query-level payload inspection not captured in the dataset.

False Positive fatigue: Random Forest produces only 30 false positives on KDD while XGBoost produces 601 — a 20× difference despite similar F1 scores. The explanation lies in each model's decision mechanism: Random Forest labels a connection as attack only when the majority of 100 trees agree, making it structurally conservative. XGBoost, trained with `scale_pos_weight = 4.13`, is explicitly incentivised to upweight the attack class, accepting more false alarms in exchange for higher recall. In a SOC context, RF's 30 FP means 30 analyst investigations per test cycle; XGBoost's 601 FP may contribute to alert fatigue even though its overall F1 is marginally higher.

Threshold optimisation (Credit Card): Sweeping the decision threshold from 0.01 to 0.99 and selecting the value that maximises F2 (which weights Recall twice as heavily as Precision) yields an optimal threshold of 0.34. At the default threshold of 0.5, XGBoost on Credit Card achieves Precision = 0.881 and Recall = 0.836. After optimisation at 0.34, Precision = 0.866 and Recall = 0.857 (F2 = 0.859). The modest Precision cost is operationally justified: the cost of a missed fraud transaction (false negative) substantially exceeds the cost of a false alarm (false positive) in financial fraud detection. This demonstrates that model selection and threshold selection are

separate decisions; the best model at 0.5 is not necessarily the best deployment choice.

6. Conclusions

6.1 Key Findings

1. XGBoost is confirmed as the strongest model under a leak-free pipeline.
2. The paper's written methodology implies data leakage (encode → scale → split), but the actual code uses scikit-learn Pipeline correctly. The discrepancy makes the work unreproducible from the paper text alone.
3. F1 alone is insufficient for highly imbalanced datasets. MCC and ROC-AUC provide essential complementary signal.
4. Rare attack subtypes (snmpgetattack) are systematically missed despite high aggregate Recall.
5. Threshold tuning is operationally critical; 0.5 default is not optimal.

6.2 Strengths of the Paper

- The research question is genuine and practically important.
- Using two datasets with different imbalance profiles provides a more robust evaluation.
- The statistical test on imputation methods is an appropriate use of significance testing.

6.3 Weaknesses

- Text/code inconsistency implying data leakage (most serious documentation failure).
- Incomplete metric suite (no MCC, no ROC-AUC).
- Unequal hyperparameter tuning across model families.
- No confidence intervals for the main model comparison.
- Poor reproducibility documentation.

6.4 Recommendation

XGBoost with `scale_pos_weight` is a strong practical baseline for imbalanced cybersecurity classification. However, the preprocessing pipeline must be corrected (split before fit), the metric suite extended to include MCC and ROC-AUC, and the decision threshold tuned to the deployment's specific FP/FN cost profile.

6.5 Lessons Learned

Reproducing a paper is more demanding than it first appears. Three lessons stood out during this project.

Data leakage is easy to introduce and hard to detect. The original paper's methodology text describes fitting the scaler before splitting — which would inflate performance metrics — but the actual code uses scikit-learn Pipeline and avoids this. The danger is real: a reader who follows the paper's written description literally would introduce leakage without realising it. Even small procedural choices — like fitting a StandardScaler on test rows — can silently invalidate a comparison.

Metric choice determines what you optimise for. Accuracy is misleading on class-imbalanced datasets. The shift to F2 (which weights recall over precision) changed the recommended decision threshold from 0.5 to 0.34, recovering hundreds of

additional true positives on KDD. A practitioner choosing a classifier purely on accuracy would arrive at a different—and operationally worse—deployment decision.

Reproducibility requires explicit documentation. The paper omits the random seed (`RANDOM_STATE = 42` is defined in the code but never stated in the paper), the named dataset variant (`kddcup_data_corrected.csv` appears in the code but not the paper), and the `requirements.txt` references a private local path for `model_utils` — though no notebook actually imports it, so it is a setup inconvenience rather than a runtime blocker. These are not minor omissions: a reader using only the paper text cannot reproduce the exact train/test split, cannot identify the dataset variant, and may waste time on a dependency that turns out not to be needed. A reproducible paper should publish code, seeds, and data provenance as first-class outputs alongside the PDF.

6.6 Suggestions for Future Improvements

Several extensions would make the work more rigorous and practically useful.

Correct the preprocessing pipeline and re-run the full comparison. All reported numbers should be recomputed with a split-first pipeline to produce a clean, leakage-free baseline. Publishing these corrected numbers would let future researchers build on a sound foundation.

Evaluate on a modern, traffic-based dataset. KDD99 and NSL-KDD are over two decades old and do not reflect current network traffic patterns or attack vectors. Replacing or supplementing them with CIC-IDS-2017 or UNSW-NB15 would test whether the classifiers generalise to realistic modern threats.

Add cost-sensitive evaluation. In a real SOC, a false negative (missed attack) carries a very different cost from a false positive (analyst alert fatigue). A cost matrix that reflects this asymmetry, together with systematic threshold optimisation, would make model selection decisions more operationally defensible.

Include confidence intervals and significance tests. Single-split metric differences between models may not be statistically meaningful. Reporting bootstrapped confidence intervals or a McNemar test would clarify which performance gaps are genuine and which are noise.

Release a fully self-contained reproduction package. Publishing the notebook, fixed random seeds, the exact dataset file, and all helper modules as a single archived repository would allow any reader to reproduce the results with a single command. This is the minimum standard for a reproducibility-focused paper.